

PropControl Billion-Dollar Billing, Credits, Pricing, and Schema Master Build Package

Unified founder package for usage-based billing, Stripe credit top-ups, Supabase ledger architecture, complete schema appendix, swarm governance, messaging costs, and production rollout.

Purpose	Single source of truth for Emergent AI agent buildout, founder review, and production handoff.
Commercial model	Base subscription tiers + prepaid credits + workflow-level usage charges + premium variants.
Primary billing stack	Stripe Checkout + webhooks + Supabase/Postgres ledger + pricing configs + usage middleware.
Supersedes	All earlier founder/billing packages. Use this package only.

Core rule: never grant credits from the frontend; only grant credits after a verified Stripe webhook and a durable ledger write.

What this package contains

- Executive strategy and pricing model
- Full PropControl usage-based pricing table
- Feature families, variants, minimum charges, and profit guardrails
- Seed tables for credit_feature_pricing and pricing_configs
- Stripe purchase flow, webhook sequence, and auto-reload rules
- Full database schema appendix including swarm tables
- RPC/database functions and ledger logic
- Billing middleware for AI, swarms, voice, vision, bot, SMS, and exports
- Frontend UX, admin ops, observability, and QA
- Built vs specified vs still-needs-QA checklist
- Copy/paste implementation brief for the Emergent AI agent

Executive strategy

PropControl should keep Growth / Pro / Team as subscription tiers for platform access and layer prepaid credits on top for all variable-cost workflows. This protects recurring revenue while preventing margin bleed from Gemini, Google Places, Street View, RentCast, Twilio, Telegram bot usage, and heavy image/vision workflows.

Commercial rules

- Charge users by workflow outcome or unit outcome, not by raw provider calls or token math.
- Keep all provider costs configurable in database pricing tables so pricing can be changed without redeploying.
- Use standard, premium, and swarm-enriched variants where the cost profile changes materially.
- Apply minimum batch charges for expensive workflows such as swarms and bulk analysis.
- Expose estimated credit usage before expensive runs and show low-balance warnings before users hit zero.
- Treat Market Swarm and Vendor Swarm as premium workflows, not included subscription features.
- Treat Telegram bot and Twilio SMS as separately metered features because their costs are distinct from model inference.

Base plans and credits

Layer	What it covers	Revenue logic
Growth / Pro / Team	Platform access, CRM, saved properties, dashboards, workflows, seats, support	Predictable MRR
Credit packs	Prepaid credits for AI/data/messages/vision/swarms	Usage revenue and margin protection
Premium variants	Heavier reasoning, bigger swarms, richer exports, larger image/vision jobs	Upsell path

Full PropControl usage-based pricing table

Recommended v1 retail basis: 1 credit = \$0.01 retail equivalent. Tune later based on real cost logs, margin by feature, and provider cost drift.

Family	Feature	Variant	Charge unit	Cred.	Min	Retail	Notes
core_analys	Rent Estimate	standard	per property	8	8	\$0.08	Single-property rent estimate with light AI formatting
core_analys	Appraisal Report	standard	per report	35	35	\$0.35	Standard valuation-style narrative report
core_analys	Appraisal Report	premium	per report	50	50	\$0.50	Heavier reasoning or map-enriched version
core_analys	Comps Explorer Run	standard	per run	12	12	\$0.12	Charge only on executed run or refresh
core_analys	AI Underwriting	standard	per report	20	20	\$0.20	Deal summary, risk/reward, scenario analysis
core_analys	AI Underwriting	premium	per report	30	30	\$0.30	Advanced reasoning or deeper scenario set
core_analys	Bulk Property Analysis	standard	per property	6	25	\$0.06	25-credit batch minimum recommended
core_analys	Bulk	swarm-	per	9	25	\$0.09	Use when

Family	Feature	Variant	Charge unit	Cred.	Min	Retail	Notes
sis	Property Analysis	enriched	property				map/swarm enrichment is included
core_analysis	Export / Report	standard	per export	5	5	\$0.05	Standard PDF/CSV/XLSX or simple report package
core_analysis	Export / Report	premium_investor_packet	per export	10	10	\$0.10	Branded investor packet / premium packaged export
pm_workflows	Service SOW + Owner Approval Email Generator	standard	per generated package	14	14	\$0.14	Scope of work plus owner-facing approval email
pm_workflows	AI Scripts Generator	standard	per script set	10	10	\$0.10	Leasing, owner, vendor, PM, or sales script set
pm_workflows	Voice-to-Text SOW Generator	standard	per converted output	12	12	\$0.12	Audio -> transcript -> structured SOW
pm_workflows	Rehab Studio SOW + ARV + ROI Generator	standard	per report	30	30	\$0.30	Combines SOW, investment math, and narrative

Family	Feature	Variant	Charge unit	Cred.	Min	Retail	Notes
vision_rehab	Visual SOW Generator	standard	per visual run	18	18	\$0.18	Vision-assisted scope generation
vision_rehab	Visual SOW Generator	premium	per visual run	25	25	\$0.25	Higher image count or stronger reasoning mode
vision_rehab	Deep Rehab Analyzer + Before/After Photos Generator	standard	per project	45	45	\$0.45	Heavy vision plus image generation workflow
vision_rehab	AI Interior Design Generator	standard	per room	16	16	\$0.16	Charge per room or concept
vision_rehab	AI Interior Design Generator	premium_3_room_pack	per pack	40	40	\$0.40	Three-room pack for better UX
vision_rehab	Premium Rehab Design Pack	premium	per pack	50	50	\$0.50	Expanded before/after and design concept package
swarms	Market Swarm Run	standard	per run	40	40	\$0.40	Base neighborhood / market lead sweep
swarms	Market Swarm Run	premium	per run	60	60	\$0.60	Larger radius, more results, richer enrichment
swarms	Vendor	standard	per run	30	30	\$0.30	Base

Family	Feature	Variant	Charge unit	Cred.	Min	Retail	Notes
	Swarm Run						vendor discovery sweep
swarms	Vendor Swarm Run	premium	per run	50	50	\$0.50	Higher result cap and more enrichment
messaging_bot	Telegram Bot Usage	standard_turn	per turn	3	3	\$0.03	Standard bot turn replacing Alex AI
messaging_bot	Telegram Bot Usage	premium_turn	per turn	5	5	\$0.05	Longer reasoning or premium workflow turn
messaging_bot	SMS Send via Twilio	standard_outbound	per message	2	2	\$0.02	Outbound SMS baseline
messaging_bot	SMS Conversation Follow-up Pack	standard_pack	per 5-message sequence	10	10	\$0.10	Workflow pack pricing

Family	Feature	Variant	Charge unit	Cred.	Min	Retail	Notes
messaging_bot	SMS + AI Reply Assist	standard_assisted_message	per message	4	4	\$0.04	AI drafting, classification, or assisted response

Pricing guardrails

- Bulk Property Analysis uses a 25-credit minimum batch charge.
- Market Swarm and Vendor Swarm use a 40-credit minimum and must show estimated credits before launch.
- Premium variants should be used when stronger models, larger result caps, richer exports, or heavier vision generation are triggered.

- Users should see action-level pricing such as Visual SOW: 18 credits, not internal provider math.

Seed tables for pricing engine

Use these tables as the starting point for database-driven pricing.

Seed table: credit_feature_pricing

feature_name	variant	credits	charge_unit	min_charge	active
Rent Estimate	standard	8	per_property	8	true
Appraisal Report	standard	35	per_report	35	true
Appraisal Report	premium	50	per_report	50	true
Comps Explorer Run	standard	12	per_run	12	true
AI Underwriting	standard	20	per_report	20	true
AI Underwriting	premium	30	per_report	30	true
Bulk Property Analysis	standard	6	per_property	25	true
Bulk Property Analysis	swarm_enriched	9	per_property	25	true
Export / Report	standard	5	per_export	5	true
Export / Report	premium_investor_packet	10	per_export	10	true
Service SOW + Owner Approval Email Generator	standard	14	per_generated_package	14	true
AI Scripts Generator	standard	10	per_script_set	10	true
Voice-to-Text SOW Generator	standard	12	per_converted_output	12	true

feature_name	variant	credits	charge_unit	min_charge	active
Rehab Studio SOW + ARV + ROI Generator	standard	30	per_report	30	true

feature_name	variant	credits	charge_unit	min_charge	active
Visual SOW Generator	standard	18	per_visual_run	18	true
Visual SOW Generator	premium	25	per_visual_run	25	true
Deep Rehab Analyzer + Before/After Photos Generator	standard	45	per_project	45	true
AI Interior Design Generator	standard	16	per_room	16	true
AI Interior Design Generator	premium_3_room_pack	40	per_pack	40	true
Premium Rehab Design Pack	premium	50	per_pack	50	true
Market Swarm Run	standard	40	per_run	40	true
Market Swarm Run	premium	60	per_run	60	true
Vendor Swarm Run	standard	30	per_run	30	true
Vendor Swarm Run	premium	50	per_run	50	true
Telegram Bot Usage	standard_turn	3	per_turn	3	true
Telegram Bot Usage	premium_turn	5	per_turn	5	true

feature_name	variant	credits	charge_unit	min_charge	active
SMS Send via Twilio	standard_outbound	2	per_message	2	true
SMS Conversation Follow-up Pack	standard_pack	10	per_5_message_sequence	10	true
SMS + AI Reply	standard_assist	4	per_message	4	true

feature_name	variant	credits	charge_unit	min_charge	active
Assist	ed_message				

Seed table: pricing_configs

provider	model_or_sku	cost_basis	markup	notes
gemini_text	gemini-2.5-flash / flash-lite / pro	per input/output token	2.5x default	Use model-specific costs and allow overrides by feature
gemini_vision	vision-capable Gemini models	per input/output token	2.5x default	Used for Visual SOW, rehab analyzer, image reasoning
google_places	Places API SKU(s)	per request/SKU	2.5x default	Used heavily in Market Swarm / Vendor Swarm
google_street_view	Street View imagery SKU(s)	per image/request	2.5x default	Used in visual qualification and swarm enrichment
google_geocoding	Geocoding SKU(s)	per request	2.5x default	Only if geocoding is in workflow
rentcast	property / rent / comps endpoints	per request	2.5x default	Used by rent estimates and comps/appraisal features
twilio_sms	US A2P / route-specific sender config	per message segment	2.5x default	Keep route-specific costs configurable
telegram_bot	bot event processing bucket	per turn	2.0x-2.5x	Use mainly for bot-specific operating cost and AI overhead
voice_transcription	transcription provider / SKU	per minute	2.5x default	Used by Voice-to-Text SOW Generator
image_generation	image generation provider / SKU	per image	2.5x default	Used by Deep Rehab and Interior Design features
export_rendering	PDF / report packaging job	per export	2.0x-2.5x	Used for packaged exports and

				investor packets
--	--	--	--	------------------

Stripe purchase flow, webhook logic, and auto-reload

1. User selects a credit package from Buy Credits UI.
2. Backend creates an internal credit_purchase_orders row before creating Stripe Checkout.
3. Backend creates Stripe Checkout Session with purchase_order_id, user_id, package_id, and credits_to_grant in metadata.
4. User completes payment in Stripe Checkout.
5. Stripe sends webhook to raw-body webhook endpoint.
6. Webhook verifies Stripe signature, persists stripe_webhook_events row, dedupes by stripe_event_id, and marks processing state.
7. Webhook calls deposit_credits(...) using an idempotency key tied to the Stripe event.
8. credit_ledger receives the deposit row and users.available_credits is incremented atomically.
9. Purchase order is marked paid; payment method can be saved for future off-session auto-reload.
10. Frontend refreshes balance and transaction history from database state.

Webhook rules

- Never grant credits from the frontend success redirect.
- Never process a Stripe event twice.
- Never update balance without a ledger row.
- Use success and async success webhook events for fulfillment.
- Treat auto-reload as phase 2 and enable it only after core billing flows are stable.

Auto-reload safeguards

- User must opt in and have a stored payment method.
- Use package-based reload amounts rather than arbitrary amounts.
- Add cooldown windows and recent-attempt checks to prevent duplicate rapid reloads.
- Log reload attempts, failures, and resulting credit grants just like manual purchases.
- Notify user on failure and do not loop retries blindly.

Full database schema appendix

The database is the source of truth for balance, credits, usage, and webhook state. Stripe is only the money movement trigger; all durable state lives in Supabase/Postgres.

users

- id (uuid, primary key)
- email (text)
- stripe_customer_id (text, nullable)
- stripe_payment_method_id (text, nullable)
- available_credits (numeric(18,6), cached fast-read balance)
- auto_reload_enabled (boolean)
- auto_reload_amount (numeric(10,2))

- low_balance_threshold (numeric(10,2))
- created_at / updated_at

pricing_configs

- id (uuid, primary key)
- provider (text)
- model_name or sku (text)
- cost_per_1k_input / cost_per_1k_output (numeric)
- flat_fee_per_request (numeric)
- markup_multiplier (numeric)
- is_active (boolean)
- notes (text)
- created_at / updated_at

credit_packages

- id (uuid, primary key)
- code (text unique)
- name (text)
- price_usd (numeric(10,2))
- credits_granted (numeric(18,6))
- is_active (boolean)
- sort_order (integer)
- created_at / updated_at

credit_purchase_orders

- id (uuid, primary key)
- user_id (uuid fk users.id)
- package_id (uuid fk credit_packages.id)
- package_code / package_name
- price_usd / credits_to_grant / currency
- status (created, checkout_open, paid, failed, canceled, refunded)
- stripe_checkout_session_id / stripe_payment_intent_id / stripe_invoice_id / stripe_charge_id
- client_reference_id (unique)
- request_idempotency_key (unique)
- paid_at / created_at / updated_at

credit_ledger

- id (uuid, primary key)
- user_id (uuid fk users.id)
- transaction_type (deposit, deduction, refund, adjustment, reversal)
- amount (numeric signed; non-zero)
- reference_type / reference_id
- description
- metadata_json
- idempotency_key (unique)

- created_at

api_usage_logs

- id (uuid, primary key)
- user_id (uuid fk users.id)
- feature_name / feature_variant
- workflow_id (nullable)
- provider_used / model_name
- input_tokens / output_tokens
- request_units
- credits_charged
- wholesale_cost / retail_cost
- is_successful
- external_request_id
- metadata_json
- created_at

stripe_webhook_events

- stripe_event_id (primary key)
- event_type
- livemode
- api_version
- object_id
- processing_status (received, processing, processed, failed, ignored)
- processing_attempts
- last_error
- payload_json
- received_at / processed_at

swarm_runs

- id (uuid, primary key)
- user_id (uuid fk users.id)
- swarm_type (market_swarm, vendor_swarm)
- status (queued, running, completed, failed, canceled)
- batch_size
- estimated_wholesale_cost / estimated_retail_cost
- actual_wholesale_cost / actual_retail_cost
- credits_reserved / credits_finalized
- leads_found / qualified_leads
- metadata_json
- created_at / updated_at / completed_at

swarm_run_steps

- id (uuid, primary key)
- swarm_run_id (uuid fk swarm_runs.id)

- step_name
- provider_used / model_name
- status
- input_tokens / output_tokens / request_units
- wholesale_cost / retail_cost
- external_request_id
- metadata_json
- created_at

swarm_leads (recommended)

- id (uuid, primary key)
- swarm_run_id (uuid fk swarm_runs.id)
- lead_type / source_provider / status
- name / address / phone / email
- score
- metadata_json
- created_at

Metadata conventions by feature type

- **SMS:** message_segments, destination_country, sender_type, carrier_fee_estimate
- **Telegram bot:** turn_length, conversation_id, intent_type
- **Voice-to-text:** audio_minutes, transcript_length, transcript_confidence
- **Vision / design:** images_processed, images_generated, vision_mode, generation_variant
- **Rehab / investment:** property_value_inputs, arv_estimate, roi_estimate, rehab_scope_size
- **Swarms:** places_requests, street_view_requests, geocode_requests, leads_found, qualified_leads, cost_breakdown_json

RPC/database functions and ledger logic

deposit_credits(...)

Verified Stripe success -> atomic credit deposit with idempotency key and ledger-first write.

deduct_credits(...)

Atomic deduction after successful usage logging; lock row, check sufficient balance, insert ledger row, decrement balance.

recalculate_user_credit_balance(...)

Support/admin repair tool that rebuilds cached balance from the ledger.

mark_stripe_webhook_processed(...)

Updates webhook state, attempts, and error details.

reserve_credits_for_workflow(...) (recommended)

Reserve estimated credits before expensive workflows such as swarms or image-heavy runs.

finalize_reserved_credits(...) (recommended)

Settle actual retail charge after completion and release unused reserve.

Ledger rules

- Never mutate historical ledger rows; corrections must be new rows using refund, reversal, or adjustment types.
- Never trust the frontend balance alone; users.available_credits is only a cache over the ledger.
- Every credit movement must carry reference_type, reference_id, description, metadata_json, and idempotency_key.
- Refunds or disputes should create reversing ledger rows and may drive an account negative if credits were already consumed.

Billing middleware rules for AI, swarms, vision, voice, bot, and SMS

11. Identify authenticated user and feature_name / feature_variant.
12. Load active pricing configs and default feature pricing rows.
13. Estimate required credits and block early if clearly insufficient.
14. For expensive workflows, reserve estimated credits before running.
15. Execute provider calls.
16. Capture raw usage metrics including tokens, request units, image counts, message segments, and minutes where applicable.
17. Calculate wholesale cost from pricing_configs.
18. Apply markup and feature pricing rules to derive retail charge and credits.
19. Insert api_usage_logs rows and, for swarms, parent/child workflow rows.
20. Deduct credits atomically or finalize reserved credits.
21. Trigger low-balance logic and, if enabled and safe, auto-reload logic.
22. Return results with user-friendly action descriptions.

Use reserve-then-settle for

- Market Swarm
- Vendor Swarm
- Deep Rehab Analyzer + Before/After Photos Generator
- Large Bulk Property Analysis runs
- Large image-heavy Visual SOW or Interior Design workflows

Frontend UX, admin ops, observability, and QA

User experience requirements

- Credit balance widget in dashboard header and billing page.
- Buy Credits page with active packages, Stripe Checkout action, and success/cancel states.
- Transaction history UI backed by credit_ledger showing deposits and deductions.
- Usage history filters by feature family and variant.
- Estimated credit cost shown before expensive actions.
- Low-balance warning when below threshold.

- Insufficient-credits modal before expensive runs.
- Auto-reload settings page with opt-in, threshold, and package amount selection.

Admin / ops requirements

- Admin billing dashboard with balances, ledger, purchase orders, usage logs, webhook history, and failed workflows.
- Manual credit adjustment tool and balance recalculation action.
- Revenue by feature, margin by feature, cost by provider, cost by swarm type, and cost-per-lead reporting.
- Top users by SMS burn, Telegram bot burn, image/vision burn, and swarm burn.
- Pricing config updates without redeploy.

Observability and alerts

- Alert on failed Stripe webhooks.
- Alert on stuck purchase orders in checkout_open state.
- Alert on deduction failures or impossible balances.
- Alert on abnormal provider cost spikes and high-cost workflow failures.
- Daily/weekly reporting on credits sold, credits consumed, wholesale cost, revenue, and gross margin by feature.

Required QA

- Successful purchase flow test
- Duplicate webhook replay test
- Success redirect without webhook test
- Insufficient balance and duplicate deduction tests
- Rapid repeated click / multi-tab race-condition test
- SMS single send and SMS pack deduction tests
- Telegram standard and premium turn deduction tests
- Visual SOW, Deep Rehab, and Interior Design deduction tests
- Voice-to-Text SOW and Rehab Studio deduction tests
- Market Swarm and Vendor Swarm standard/premium deduction tests
- Reserve-then-settle correctness tests
- Ledger recalculation repair test

Built vs specified vs still-needs-QA checklist

Area	Status in this package	What still has to happen
Billing model + credit packs	SPECIFIED	Finalize production values and ship
Database schema + functions	SPECIFIED	Run migrations in staging and production
Stripe checkout purchase flow	SPECIFIED	Build endpoint and test end-to-end

Webhook idempotency + atomic deposit	SPECIFIED	Implement webhook route and retry handling
Transaction history UI	SPECIFIED	Build frontend screen and verify correctness
Low balance warning	SPECIFIED	Wire frontend threshold logic
Auto-reload	SPECIFIED (phase 2)	Implement only after core billing proves stable
Swarm governance + reserve/settle	SPECIFIED	Implement workflow reservations/finalization
SMS / Telegram pricing and logging	SPECIFIED	Implement provider-specific logging and tests
Admin dashboard + observability	SPECIFIED	Build internal tools and alerts

Build order for the agent

23. Run schema migration in staging.
24. Seed pricing_configs, credit_packages, and credit_feature_pricing.
25. Implement deposit_credits and deduct_credits integrations.
26. Build purchase order creation endpoint and Stripe Checkout endpoint.
27. Build raw-body Stripe webhook endpoint with idempotency and ledger deposit.
28. Ship billing page, balance widget, and transaction history UI.
29. Wrap Rent Estimate, Appraisal Report, Comps Explorer, AI Underwriting, and Export / Report in billing middleware.
30. Wrap Service SOW, Visual SOW, Telegram Bot Usage, and SMS usage.
31. Wrap Voice-to-Text, Rehab Studio, Deep Rehab Analyzer, and Interior Design features.
32. Wrap Market Swarm and Vendor Swarm using reserve-then-settle.
33. Add admin dashboard, alerts, and margin reporting.
34. Enable auto-reload only after the full billing system is stable.
35. Run staging QA, soft launch, review margins, then broaden release.

Copy/paste implementation brief for Emergent AI agent

Build PropControl using this document as the single source of truth.

Commercial model: keep Growth / Pro / Team as base subscription tiers for platform access and use prepaid credits for all premium AI, data, messaging, vision, and swarm usage.

Use the default credit table in this package as the starting retail table and keep provider pricing configurable in database tables.

Use Supabase/Postgres as the source of truth. Stripe Checkout handles credit purchases. Verified Stripe webhooks trigger atomic deposit_credits ledger writes. Never grant credits from the frontend success redirect.

Every premium feature must pass through one shared billing middleware.

Market Swarm and Vendor Swarm must be credit-gated and cost-governed.

Use reserve-then-settle for large swarm and image-heavy workflows.

Log provider usage in `api_usage_logs` and `swarm child` tables.

Build user-facing balance, transaction history, low-balance warnings, and action-cost labels.

Build admin tools for ledger review, webhook review, pricing updates, and balance repair.

Treat auto-reload as phase 2 after the core billing system is stable.

Do not deviate from the schema, ledger rules, or webhook rules in this package without explicit founder approval.